

Compte Rendu du TP MLOps

Chapitre 2 : Pipelines agiles, versionnement et orchestration

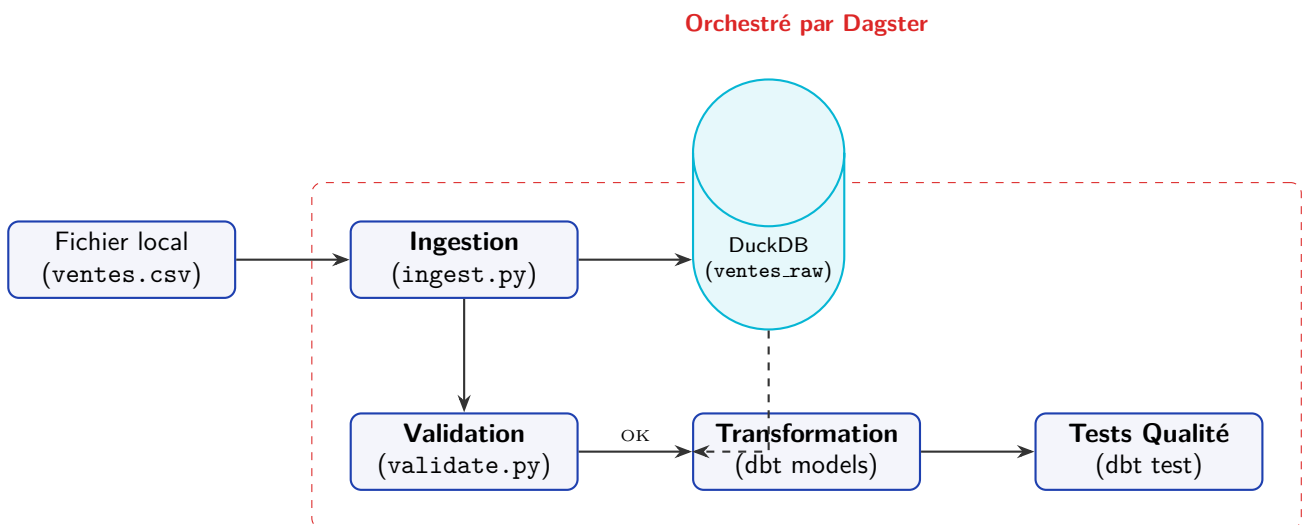
Module : MLOps & DataOps

Réalisé par :
Ayman Chabbaki
Groupe 1

Objectif du TP

Ce TP a pour but de transformer un workflow de données manuel (basé sur des fichiers isolés) en un pipeline de données structuré et industrialisable. Nous avons mis en place une architecture modulaire utilisant **DuckDB** pour le stockage local, **dbt** pour les transformations analytiques, **Dagster** pour l'orchestration du Graphe à Graphe Orienté Acyclique (DAG), et **Git / GitHub Actions** pour le versionnement et l'intégration continue (CI).

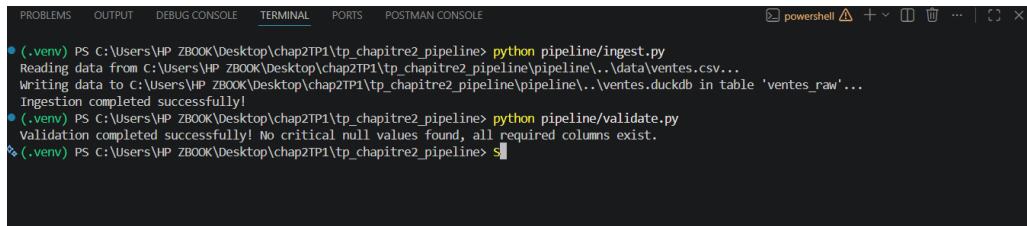
1. Schéma de l'architecture du Pipeline



2. Exécution et Captures d'écran

2.1. Ingestion et Validation

Le script `ingest.py` a chargé les données dans DuckDB, suivi de `validate.py` qui a vérifié la présence des colonnes requises et l'absence de valeurs nulles critiques.

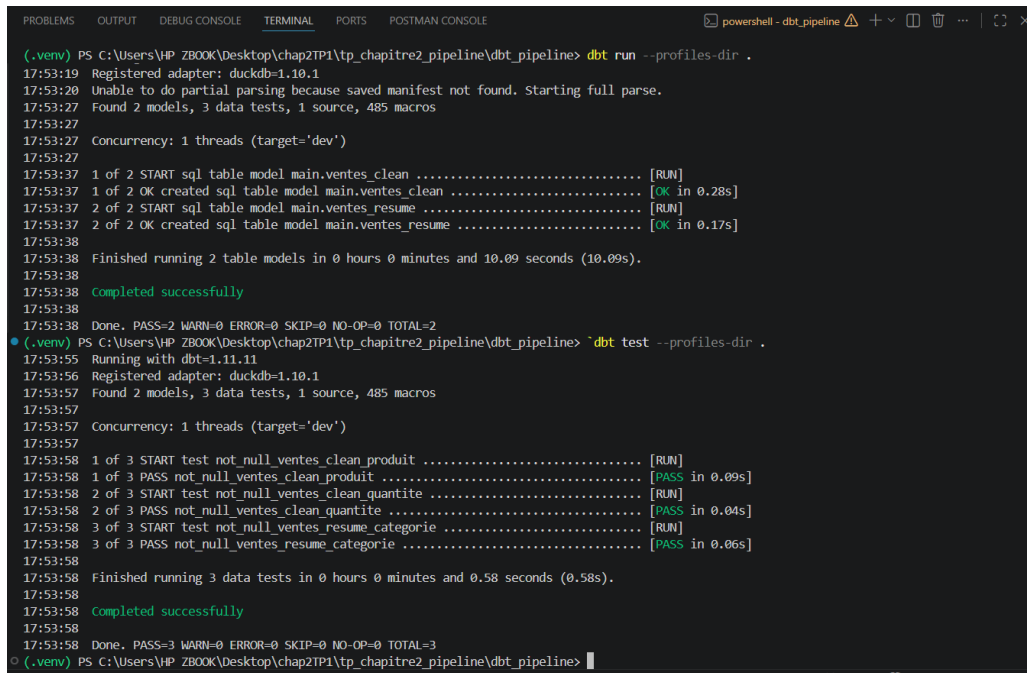


```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
(.venv) PS C:\Users\HP_ZBOOK\Desktop\chap2TP1\tp_chapitre2_pipeline> python pipeline/ingest.py
Reading data from C:\Users\HP_ZBOOK\Desktop\chap2TP1\tp_chapitre2_pipeline\pipeline\..\data\ventes.csv...
Writing data to C:\Users\HP_ZBOOK\Desktop\chap2TP1\tp_chapitre2_pipeline\pipeline\..\ventes.duckdb in table 'ventes_raw'...
Ingestion completed successfully!
(.venv) PS C:\Users\HP_ZBOOK\Desktop\chap2TP1\tp_chapitre2_pipeline> python pipeline/validate.py
Validation completed successfully! No critical null values found, all required columns exist.
(.venv) PS C:\Users\HP_ZBOOK\Desktop\chap2TP1\tp_chapitre2_pipeline> $
```

Figure 1: Exécution réussie de l'ingestion et de la validation Python.

2.2. Transformation et Tests avec dbt

Les modèles SQL ont permis de nettoyer les données et de générer un agrégat du chiffre d'affaires, validés ensuite par les tests de schéma `not null`.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
(.venv) PS C:\Users\HP_ZBOOK\Desktop\chap2TP1\tp_chapitre2_pipeline\dbt_pipeline> dbt run --profiles-dir .
17:53:19 Registered adapter: duckdb-1.10.1
17:53:20 Unable to do partial parsing because saved manifest not found. Starting full parse.
17:53:22 Found 2 models, 3 data tests, 1 source, 485 macros
17:53:27 Concurrency: 1 threads (target='dev')
17:53:27
17:53:37 1 of 2 START sql table model main.ventes_clean ..... [RUN]
17:53:37 1 of 2 OK created sql table model main.ventes_clean ..... [OK in 0.28s]
17:53:37 2 of 2 START sql table model main.ventes_resume ..... [RUN]
17:53:37 2 of 2 OK created sql table model main.ventes_resume ..... [OK in 0.17s]
17:53:38 Finished running 2 table models in 0 hours 0 minutes and 10.09 seconds (10.09s).
17:53:38 Completed successfully
17:53:38 Done, PASS=2 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=2
(.venv) PS C:\Users\HP_ZBOOK\Desktop\chap2TP1\tp_chapitre2_pipeline\dbt_pipeline> dbt test --profiles-dir .
17:53:55 Running with dbt=1.11.11
17:53:56 Registered adapter: duckdb-1.10.1
17:53:57 Found 2 models, 3 data tests, 1 source, 485 macros
17:53:57 Concurrency: 1 threads (target='dev')
17:53:57
17:53:58 1 of 3 START test not_null_ventes_clean_produit ..... [RUN]
17:53:58 1 of 3 PASS not_null_ventes_clean_produit ..... [PASS in 0.09s]
17:53:58 2 of 3 START test not_null_ventes_clean_quantite ..... [RUN]
17:53:58 2 of 3 PASS not_null_ventes_clean_quantite ..... [PASS in 0.04s]
17:53:58 3 of 3 START test not_null_ventes_resume_categorie ..... [RUN]
17:53:58 3 of 3 PASS not_null_ventes_resume_categorie ..... [PASS in 0.06s]
17:53:58 Finished running 3 data tests in 0 hours 0 minutes and 0.58 seconds (0.58s).
17:53:58 Completed successfully
17:53:58 Done, PASS=3 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=3
(.venv) PS C:\Users\HP_ZBOOK\Desktop\chap2TP1\tp_chapitre2_pipeline\dbt_pipeline> 
```

Figure 2: Génération des tables et succès des tests de qualité dbt.

2.3. Orchestration Globale (Dagster)

Dagster a permis de lier ces scripts Python isolés au sein d'un seul workflow cohérent, garantissant l'ordre d'exécution.

2.4. Historique de Versionnement (Git)

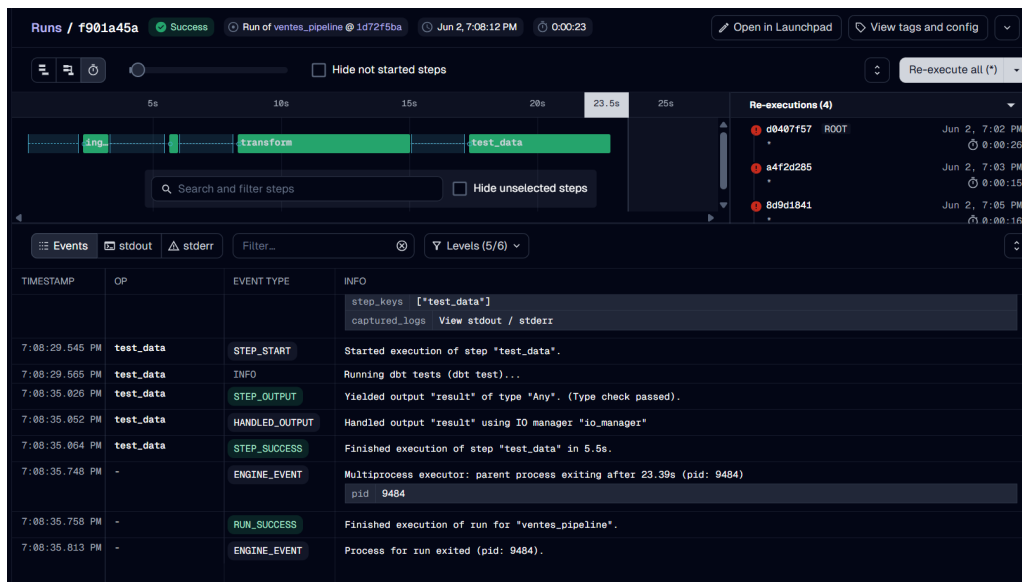


Figure 3: Vue de l'exécution du DAG dans l'interface graphique de Dagster.

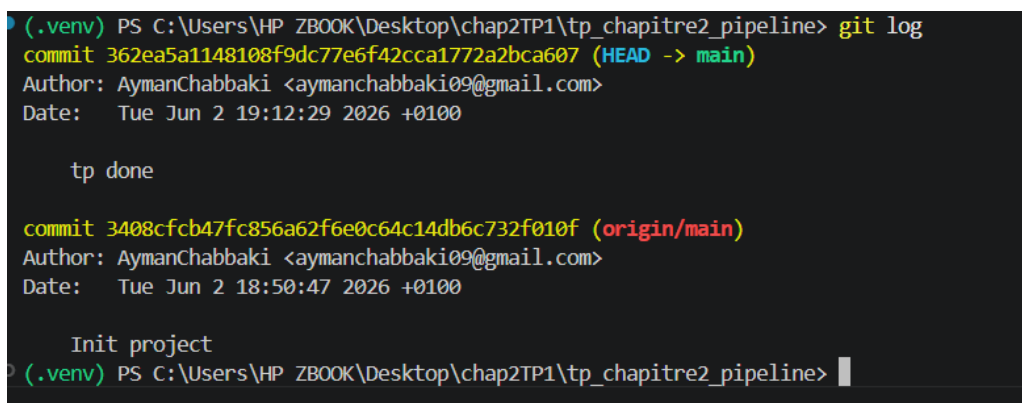


Figure 4: Historique des commits Git illustrant le versionnement du code.

3. Réponses aux questions

1. En quoi ce pipeline est-il plus structuré qu'un notebook unique ?

Contrairement à un notebook où l'état global est partagé et l'exécution dépend du clic humain, ce pipeline favorise la *modularité* (séparation des responsabilités : ingestion, validation, transformation). Chaque composant peut être testé unitairement, versionné proprement (les notebooks génèrent des conflits Git complexes), et orchestré automatiquement de manière reproductible.

2. Quel est le rôle de Git dans ce projet ?

Git agit comme le système nerveux du projet. Il permet de tracer l'historique des modifications du code source, de faciliter la collaboration en équipe (branches), d'offrir un mécanisme de retour en arrière (rollback) en cas d'erreur, et de servir de déclencheur pour les pipelines CI/CD (GitHub Actions).

3. Quel est le rôle de requirements.txt ?

Il permet de figer l'environnement Python. Il garantit la *reproductibilité* absolue : tout développeur (ou serveur CI, ou conteneur Docker) qui exécute `pip install -r requirements.txt` disposera exactement des mêmes versions des bibliothèques (dbt, dagster, duckdb), évitant ainsi le problème du "ça marche sur ma machine".

4. Pourquoi ajouter une étape validate.py ?

C'est un "Data Quality Gate" (point de contrôle qualité). Il permet d'échouer rapidement (*fail*

fast) avant de consommer des ressources de calcul coûteuses dans la phase de transformation. Si le schéma d'entrée change ou si des données corrompues arrivent, le pipeline s'arrête net, évitant de polluer les tables ou les modèles de Machine Learning avants.

5. **Quel est le rôle de `dbt test` ?**

Il permet d'appliquer une logique de Software Engineering (tests unitaires) aux données SQL. Au lieu de simplement vérifier si la requête s'exécute, **dbt test** s'assure que les données *résultantes* respectent des règles métiers (ex: l'unicité d'une clé primaire, l'absence de valeurs nulles, ou des relations référentielles).

6. **Qu'apporte Dagster par rapport à une exécution manuelle ?**

Dagster apporte l'automatisation et la résilience. Il gère l'ordre d'exécution (le DAG), garantit qu'une tâche ne démarre que si la précédente a réussi, offre une UI pour le monitoring, gère les reprises sur erreur (*retries*), et permet la planification (CRON) indépendamment d'une intervention humaine.

7. **Qu'apporte la CI (Intégration Continue) ?**

La CI automatisée par GitHub Actions agit comme un filet de sécurité. À chaque "push" de code, un serveur distant vérifie instantanément si la syntaxe Python est correcte et si les dépendances s'installent bien. Cela empêche l'intégration de code "cassé" dans la branche principale (**main**).

8. **Que manque-t-il encore pour un pipeline pleinement industrialisé ?**

Pour être totalement "production-ready", il manquerait :

- **Conteneurisation globale** : Utiliser **Docker** pour toute l'application, pas juste **venv**.
- **Versionnement de la donnée** : Remplacer le CSV local par un bucket cloud (S3/GCS) et utiliser un outil comme **DVC**.
- **Déploiement Continu (CD)** : Déployer automatiquement le code testé sur un serveur distant (Cloud).
- **Alerting** : Connecter Dagster à Slack/Email pour alerter l'équipe en cas de plantage du job en production.